

- SGE Atlântida — Project Documentation
 - 1. Project Overview
 - 2. Mandatory Features (from official document)
 - Features REMOVED from scope (axed by department)
 - 3. Tech Stack
 - Current Stack
 - What is NOT in this stack (and why)
 - 4. Project Structure
 - 5. Environment Variables (.env)
 - 6. Key Configuration Files
 - middleware.ts
 - lib/auth.ts
 - 7. Database Schema (37 modelos + 17 enums)
 - Enums
 - Core Tables
 - Nota Model (full structure)
 - Grade Calculation Logic
 - Edge cases do cálculo
 - 8. Payment System
 - Sistema de Preços Hierárquico
 - Sistema de Bolsas
 - Valores Padrão Globais (Fallback via ConfiguracaoTaxas)
 - Monthly Propina Flow
 - Reference Format
 - Duplicate Payment Protection
 - Payment confirmation security
 - What gets BLOCKED when payment is Pendente
 - What stays ACCESSIBLE when Pendente
 - Receptionist handles ONLY
 - Document Printing Limit
 - 9. Business Logic Rules
 - Year Progression / Re-enrollment
 - Grade System
 - Grade Access Control
 - Grade Computing (from CursoDisciplina)
 - Average Calculation
 - Conflitos de Professor

- Monografia Workflow
- Monografia Access Rules
- Horário de Aulas
- Plano de Provas
- Sistema de Gestão de Administradores
- Gestão de Orientadores Admin
- Documentos / Assinaturas / Verificação
- 10. Test Credentials (after running seed)
- 11. Seed Data
 - Currently creates
- 12. Commands Reference
- 13. Module Build Plan
 - Module 1 — Student Portal  COMPLETO
 - Visão Geral (/estudante)
 - Notas (/estudante/notas)
 - Pagamentos (/estudante/pagamentos)
 - Monografia (/estudante/monografia)
 - Certificados (/estudante/certificados)
 - Horário (/estudante/horario)
 - Serviços (/estudante/servicos)
 - Module 2 — Grade Management  COMPLETO
 - Plano Escolar (Horário + Provas)
 - Module 3 — Monografia Workflow  COMPLETO
 - Estudante
 - Orientador
 - Gestor
 - Module 4 — Recepcionista  COMPLETO
 - Module 5 — Admin  100% COMPLETO
- 14. Extra Features (Post-Completion)
 - Rematrícula
 - Documentos e Assinaturas
 - Configuração de Preços por Curso
 - Serviços Dinâmicos
 - Sistema de Bolsas
 - Gestão Centralizada (Admin + Gestor)
- 15. Component Architecture
 - DashboardLayout (shared by all roles)
 - Pattern for each page

- [AuditLog helper \(lib/audit.ts\)](#)
- [Grade calculation helper \(lib/notas.ts\)](#)
- 16. UI Design System
 - [Status Colours](#)
- 17. Dificuldades e Lições Aprendidas
 -  [Next.js 15 App Router — params é uma Promise](#)
 -  [Prisma Decimal — Bug Number\(null\) = 0](#)
 -  [TurboPack Hydration Errors](#)
 -  [Timezone UTC shift \(Africa/Luanda UTC+1\)](#)
 -  [Sidebar nav overflow](#)
 -  [JWT não atualiza após edição de perfil](#)
 -  [Disciplinas aparecem no ano/semestre errado](#)
 -  [Edição de notas — valores sobrescritos com null](#)
- 18. Current State
 - [Completed](#)

SGE Atlântida — Project Documentation

Last updated: May 16, 2026 **Status:** 🎉 100% COMPLETO! Todos os 5 Módulos + Funcionalidades Extra implementados e funcionais **Production URL:** <https://sge-atl-ntida.vercel.app/> (Neon PostgreSQL)

1. Project Overview

What we are building: A Sistema de Gestão Escolar (SGE) — Academic Management System — for Instituto Superior Politécnico Atlântida (ISP Atlântida), Angola. This is a final semester capstone project (TTP) for 4th year Computer Engineering students, serving as the evaluation for **ALL** second semester subjects.

Official document reference: Circular from Eng. Walter A. A. S. Neto, Director of the Organic Unit of Computer Engineering and Civil, dated 30/03/2026.

2. Mandatory Features (from official

document)

1. **Full Student Self-Service Portal** — student handles enrollment, fees, certificates, transcripts online without going to the secretary
2. **Multicaixa Express Payment Integration** — Angola-specific payment system for monthly propinas (tuition fees)
3. **Credit-Based Academic Control** — students enroll per subject, year progression based on passing all subjects
4. **Security and Access Levels** — audit logging: who changed a grade, when, and from which IP

Features REMOVED from scope (axed by department)

- AI retention prediction
- Blockchain diploma verification
- Micro-certifications and badges
- Skills portfolio / Dynamic CV

3. Tech Stack

Current Stack

Layer	Technology	Why
Frontend	Next.js 15 + TypeScript	File-based routing, React components, one codebase
Styling	Tailwind CSS	Utility-first, no separate CSS files
Auth	NextAuth JWT (custom refresh-jwt endpoint)	JWT strategy with role-based protection, refresh endpoint to update session after profile edit
Backend	Next.js API Routes	Same codebase as frontend, no separate server needed

Layer	Technology	Why
ORM	Prisma 6	Type-safe database queries, schema-first, auto migrations
Database	PostgreSQL (Neon)	Cloud-hosted PostgreSQL with pooled + direct connection
DB Driver	@prisma/adapters-pg	Required by Prisma 6 for direct PostgreSQL connection
Password	bcryptjs	Secure password hashing
PDF Generation	@react-pdf/renderer	React components for generating professional certificates & declarations
Calendar	react-day-picker + date-fns (locale ptBR)	Date picker with Portuguese locale
Runtime	Node.js v22	Required for Next.js

What is NOT in this stack (and why)

- **No Python/FastAPI** — was needed for AI model, AI was removed from scope
- **No Redis** — NextAuth handles sessions natively, no cache layer needed
- **No blockchain** — removed from scope
- **No WSL required** — project runs on Windows native (WSL had network issues during setup)
- **No MinIO** — file storage uses local filesystem (uploads/ directory)
- **No Docker** — everything runs natively

4. Project Structure

```
sge-atlantida/  
├─ app/  
│   └─ admin/  
│       └─ page.tsx  
monografias, pagamentos  
│   └─ adminNav.ts  
navegação admin  
│   └─ estudantes/  
│       └─ page.tsx
```

← Dashboard: stats, total students,

← Fonte única de verdade para

		└─ EstudantesAdminDashboard.tsx	← CRUD completo + notas editáveis +
pagamentos			
		└─ orientadores/	
		└─ page.tsx	
		└─ OrientadoresAdminDashboard.tsx	← Com pesquisa, filtros e gestão de
gestores			
		└─ recepcionistas/	
		└─ page.tsx	
		└─ RecepcionistasDashboard.tsx	← CRUD recepcionistas
		└─ admins/	
		└─ page.tsx	
		└─ AdminsDashboard.tsx	← Gestão de administradores
		└─ cursos/	
		└─ page.tsx	
		└─ CursosAdminDashboard.tsx	← CRUD cursos + preços por ano
		└─ disciplinas/	
		└─ page.tsx	
		└─ DisciplinasAdminDashboard.tsx	← CRUD disciplinas + dispensa config
		└─ departamentos/	
		└─ page.tsx	
		└─ DepartamentosAdminDashboard.tsx	← CRUD departamentos + gestor
		└─ pagamentos/	
		└─ page.tsx	
		└─ PagamentosAdminDashboard.tsx	← Todos pagamentos com filtros
		└─ precos/	
		└─ page.tsx	
		└─ PrecosAdminDashboard.tsx	← Preços globais + serviços +
horário	config		
		└─ audit/	
		└─ page.tsx	
		└─ AuditDashboard.tsx	← Audit log com filtros avançados
		└─ sistema/	
		└─ ano-lectivo/	
		└─ page.tsx	
		└─ AnoLectivoDashboard.tsx	← Gestão ano lectivo + matrícula
		└─ semestre/	
		└─ page.tsx	
		└─ SemestreDashboard.tsx	← Controlo de semestre actual
		└─ finalistas/	
		└─ page.tsx	
		└─ FinalistasDashboard.tsx	← Finalização de estudantes
		└─ layout-documentos/	
		└─ page.tsx	
		└─ LayoutDocumentosDashboard.tsx	← Layouts de documentos PDF
		└─ assinaturas/	
		└─ page.tsx	← Upload assinaturas (presidente,
gestor, diretor)			
		└─ registos/	
		└─ page.tsx	
		└─ RegistosDashboard.tsx	← Registos/pipeline manuais
		└─ simulador/	
		└─ route.ts	← Simulador de data do sistema
		└─ estudante/	
		└─ page.tsx	
		└─ EstudanteDashboard.tsx	← Visão geral: perfil, disciplinas,
propina			
		└─ orientador/	

	└─ page.tsx	
	└─ OrientadorDashboard.tsx	← Pedir orientação, ver estado
	└─ notas/	
	└─ page.tsx	
	└─ NotasDashboard.tsx	← Histórico completo de notas
	└─ horario/	
	└─ page.tsx	
	└─ HorarioDashboard.tsx	← Horário semanal do estudante
	└─ pagamentos/	
	└─ page.tsx	
	└─ PagamentosDashboard.tsx	← Pagamentos, Multicaixa confirmação
	└─ monografia/	
	└─ page.tsx	
	└─ MonografiaDashboard.tsx	← Pré-projecto + monografia + upload
	└─ certificados/	
	└─ page.tsx	
	└─ CertificadosDashboard.tsx	← Pedir/descarregar certificados
	└─ declaracao/	
	└─ page.tsx	
	└─ DeclaracaoDashboard.tsx	← Declarações académicas
	└─ servicos/	
	└─ page.tsx	
	└─ ServicosDashboard.tsx	← Comprar serviços
	└─ orientador/	
	└─ page.tsx	
	└─ OrientadorDashboard.tsx	← Dashboard: disciplinas,
solicitações		
	└─ solicitacoes/	
	└─ page.tsx	← Pedidos de orientação pendentes
	└─ monografias/	
	└─ page.tsx	← Monografias dos seus estudantes
	└─ recepcionista/	
	└─ page.tsx	← Dashboard + pesquisa de estudante
	└─ estudante/	
	└─ [id]/page.tsx	← Detalhe: pagamentos, documentos,
emitir		
	└─ gestor/	
	└─ page.tsx	
	└─ GestorDashboard.tsx	← Visão geral do departamento
	└─ gestorNav.ts	← Navegação centralizada com dropdown
	└─ estudantes/	
	└─ page.tsx	
	└─ EstudantesDashboard.tsx	← Lista estudantes + percurso
académico		
	└─ disciplinas/	
	└─ page.tsx	
	└─ DisciplinasDashboard.tsx	← Disciplinas + edição inline de
notas		
	└─ horario/	
	└─ page.tsx	
	└─ HorarioDashboard.tsx	← Grade semanal + impressão
	└─ plano-provas/	
	└─ page.tsx	
	└─ PlanoProvasDashboard.tsx	← Calendário de provas + impressão
	└─ curriculo/	
	└─ page.tsx	
	└─ CurriculoDashboard.tsx	← Gestão de currículo do curso

└─ monografias/	
└─ page.tsx	
└─ MonografiasDashboard.tsx	← Todas monografias do departamento
└─ pipeline-monografia/	
└─ page.tsx	
└─ PipelineDashboard.tsx	← Pipeline visual de monografias
└─ verificar/	
└─ [id]/page.tsx	← Verificação de documentos
└─ login/	
└─ page.tsx	
└─ components/	
└─ DashboardLayout.tsx	← Layout partilhado (sidebar + topbar)
└─ SessionProvider.tsx	
└─ DatePickerPT.tsx	← DatePicker com locale português
└─ DeclaracaoPDF.tsx	← PDF declaração académica
└─ CertificadoDisciplinasPDF.tsx	← PDF certificado de disciplinas
└─ CertificadoConclusaoPDF.tsx	← PDF certificado de conclusão
└─ FacturaPDF.tsx	← PDF factura
└─ FacturaTalao.tsx	← PDF talão de factura
└─ api/	
└─ auth/	
└─ [...nextauth]/route.ts	
└─ refresh-jwt/route.ts	← Recarrega dados do DB e atualiza JWT
└─ auth/refresh-jwt/route.ts	
└─ estudante/	
└─ perfil/route.ts	
└─ disciplinas/route.ts	
└─ notas/route.ts	
└─ notas-resumo/route.ts	
└─ horario/route.ts	
└─ plano-provas/route.ts	
└─ monografia/	
└─ route.ts	
└─ upload/route.ts	
└─ premonografia/route.ts	
└─ solicitacao-orientacao/route.ts	
└─ certificados/	
└─ route.ts	
└─ pedir/route.ts	
└─ declaracao/	
└─ route.ts	
└─ pdf/route.ts	
└─ [id]/pdf/route.ts	
└─ certificado/	
└─ conclusao/pdf/route.ts	
└─ disciplinas/pdf/route.ts	
└─ pagamentos/	
└─ route.ts	
└─ confirmar/route.ts	
└─ servicos/	
└─ route.ts	
└─ confirmar/route.ts	
└─ notas/route.ts	← POST criar/actualizar nota
└─ orientador/	
└─ disciplinas/	
└─ route.ts	
└─ [id]/estudantes/route.ts	


```
├─ solicitacoes/
│   └─ route.ts
│     └─ [id]/route.ts
├─ monografias/
│   └─ route.ts
│     └─ [id]/route.ts
├─ download/route.ts
├─ premonografia/[id]/route.ts
├─ gestor/
│   └─ resumo/route.ts
├─ estudantes/
│   └─ route.ts
│     └─ [id]/route.ts
│       └─ notas/[id_nota]/route.ts
├─ disciplinas/
│   └─ route.ts
│   └─ [id]/estudantes/route.ts
│   └─ [id]/professores/route.ts
│   └─ disponiveis/route.ts
├─ orientadores/
│   └─ route.ts
│   └─ lista/route.ts
├─ monografias/
│   └─ route.ts
│   └─ [id]/route.ts
├─ premonografia/[id]/route.ts
├─ horario/route.ts
├─ plano-provas/route.ts
├─ periodo-provas/route.ts
├─ curriculo/route.ts
├─ cursos/route.ts
├─ pipeline-consistency/route.ts
├─ pipeline-report/route.ts
├─ recepcionista/
│   └─ me/route.ts
├─ estudante/
│   └─ route.ts
│   └─ [id]/route.ts
│   └─ emitir/route.ts
│   └─ auditar/
│     └─ impressao/route.ts
│     └─ contagem/route.ts
├─ factura/entregar/route.ts
├─ estudantes/[id]/declaracao/pdf/route.ts
├─ certificados/[id]/pdf/route.ts
├─ admin/
│   └─ estudantes/
│     └─ route.ts
│     └─ [id]/route.ts
├─ orientadores/route.ts
├─ cursos/
│   └─ route.ts
│   └─ [id]/route.ts
├─ departamentos/
│   └─ route.ts
│   └─ [id]/route.ts
├─ disciplinas/route.ts
```

← Pesquisa de estudantes

← Emitir facturas

```

├── pagamentos/route.ts
├── propinas/[id]/route.ts
├── recepcionistas/
│   ├── route.ts
│   └── [id]/route.ts
├── admins/
│   ├── route.ts
│   └── [id]/route.ts
├── stats/route.ts
├── config/
│   ├── taxas/route.ts
│   ├── servicos/route.ts
│   └── precos-curso/route.ts
├── gerar-propinas/route.ts
├── gerar-propinas-falta/route.ts
├── assinaturas/
│   ├── presidente/route.ts
│   ├── gestor/route.ts
│   └── diretor/route.ts
├── layout-documentos/
│   ├── route.ts
│   └── [tipo]/route.ts
├── sistema/
│   ├── config/route.ts
│   ├── finalistas/route.ts
│   ├── semestre/route.ts
│   ├── registros/route.ts
│   ├── simulador/route.ts
│   └── suspender-sem-rematricula/route.ts
├── verificar/
│   └── [id]/route.ts
├── audit/route.ts
├── layout.tsx
├── globals.css
├── page.tsx
├── lib/
│   ├── auth.ts
│   ├── prisma.ts
│   ├── audit.ts
│   ├── notas.ts
│   └── sistema.ts
├── getSystemDate()
├── precos.ts
├── getPrecosGlobais() (com bolsas)
├── propinas.ts
├── multa-atraso.ts
├── reenrollment.ts
├── suspenderEstudantesSemRematricula()
├── atribuirDisciplinas.ts
├── verificarConflitos.ts
├── provas)
├── servicos-tipos.ts
├── layout-defaults.ts
├── phone.ts
├── minio.ts
├── not used/
├── prisma/

```

		schema.prisma	← 37 modelos + 17 enums
		seed.ts	← Seed principal (simplificado)
		seed-base.ts	← Dados base (departamentos, cursos,
	admin)		
		seed-estudantes.ts	← Estudantes + notas
		seed-propinas.ts	← Propinas/mensalidades
		seed-registos.ts	← Registos académicos
		seed-semester.ts	← Config semestre
		seed-rematricula.ts	← Rematrícula
		seed-snapshot-finalizados.ts	← Snapshot de estudantes finalizados
		seed-fix-ano-lectivo.ts	
		seed-fix-curriculo.ts	
		seed-fix-estudantes-disciplinas.ts	
		seed-limpar-estudantes.ts	
		seed-limpar-notas-futuras.ts	
		fix-curriculo-duplicado.ts	
		fix-disciplinas-retroativo.ts	
		fix-estudantes.ts	
		fix-notas-arredondamento.ts	
		fix-notas-incorretas.ts	
		fix-notas-limpeza.ts	
		fix-propinas-transfer.ts	
		migrations/	
		uploads/	
		monografias/{student_id}/	
		premonografias/{student_id}/	
		docs/	
		audit-pipeline-monografia.md	
		credenciais-seed.md	
		deploy-checkpoint.md	
		deploy-guide.md	
		layout-documentos-pdf.md	
		pagamentos-audit-fixes.md	
		seed-sistema-simulado.md	
		sistema-cores-acessibilidade.md	
		middleware.ts	
		.env	
		next.config.ts	
		package.json	
		tsconfig.json	

5. Environment Variables (.env)

```
# SGE Atlântida - Environment Variables

# Neon Pooled URL (for runtime - app uses this)
DATABASE_URL=postgresql://user:password@ep-xxxx-pooler.c.region.aws.neon.tech/db?
sslmode=require

# Neon Direct URL (for Prisma migrations)
```

```
DIRECT_URL=postgresql://user:password@ep-xxxx.c.region.aws.neon.tech/db?
sslmode=require
```

```
NEXTAUTH_SECRET=sge-atlantida-secret-key-2026
NEXTAUTH_URL=http://localhost:3000
```

6. Key Configuration Files

middleware.ts

```
import { withAuth } from "next-auth/middleware"
import { NextResponse } from "next/server"

export default withAuth(
  function middleware(req) {
    const token = req.nextauth.token
    const path = req.nextUrl.pathname

    if (path.startsWith("/admin") && token?.role !== "admin")
      return NextResponse.redirect(new URL("/login", req.url))
    if (path.startsWith("/estudante") && token?.role !== "estudante")
      return NextResponse.redirect(new URL("/login", req.url))
    if (path.startsWith("/orientador") && token?.role !== "orientador")
      return NextResponse.redirect(new URL("/login", req.url))
    if (path.startsWith("/repcionista") && token?.role !== "repcionista")
      return NextResponse.redirect(new URL("/login", req.url))
    // Gestor (orientador com e_gestor=true) ou admin podem acessar /gestor
    const isGestor = token?.role === "orientador" && token?.e_gestor === true
    const isAdmin = token?.role === "admin"
    if (path.startsWith("/gestor") && !isGestor && !isAdmin)
      return NextResponse.redirect(new URL("/login", req.url))
  },
  {
    callbacks: {
      authorized: ({ token }) => !!token,
    },
  }
)

export const config = {
  matcher: [
    "/admin/:path*",
    "/estudante/:path*",
    "/orientador/:path*",
    "/repcionista/:path*",
    "/gestor/:path*",
    "/dashboard/:path*",
    "/verificar/:path*",
  ],
}
```

```
],  
}
```

lib/auth.ts

Configura NextAuth com estratégia JWT, incluindo `nome_usuario`, `role`, `e_gestor` no token e session callbacks. Inclui endpoint `/api/auth/refresh-jwt` para recarregar dados do DB e actualizar a sessão após edição de perfil.

7. Database Schema (37 modelos + 17 enums)

Enums

Enum	Values
TipoUsuario	admin estudante orientador recepcionista
EstadoEstudante	EmCurso Finalizado Desistente Suspendido
EstadoMonografia	Submetida EmRevisao Aprovada ParaDefender Defendida Rejeitada
EstadoPremonografia	Proposto Aprovado Reprovado Cancelado
EstadoSolicitacao	Pendente Aceite Recusado Cancelado
EstadoFactura	Pendente Pago Atrasado
EstadoNotaCobranca	Pendente Pago Negociado
FormaPagamento	Multicaixa Transferencia Dinheiro
TipoCertificado	Conclusao Disciplina Participacao
StatusCertificado	Solicitado EmPreparacao ProntoParaLevantamento Entregue
TurnoRecepcionista	Manha Tarde Noite
Semestre	S1 S2

Enum	Values
TipoAvaliacao	Normal Recurso Especial
TipoBolsa	Nenhuma Cinquenta Cem
EmitidoPor	sistema estudante
TipoFaseAvaliacao	AC PP1 TTP_PP2 Exame Recurso_ExameEspecial
TipoDocumentoLayout	CertificadoConclusao DeclaracaoAcademica CertificadoDisciplinas

Core Tables

Model	Description
Usuario	Base user — all roles (admin, estudante, orientador, recepcionista) share this
Admin	Admin profile (nome_completo, telefone)
Orientador	Professor profile (nome_completo, especialidade, e_gestor, id_departamento)
Estudante	Student profile (nome_completo, numero_estudante, id_curso, ano_current, turno, estado, tipo_bolsa)
Recepcionista	Receptionist profile (nome_completo, turno)
Departamento	Academic department
Curso	Course (nome, duracao_anos, turnos)
Disciplina	Subject (nome, codigo, creditos, id_departamento, tem_dispenza, nota_dispenza)
CursoDisciplina	N:N Curso→Disciplina with ano_curricular + semestre
ProfessorDisciplina	Teacher assignment to discipline (id_usuario, id_disciplina, ano_lectivo,

Model	Description
	semestre)
Nota	Full grade record (AC1-3, TTP, PP1-2, Exame, Recurso, Especial, nota_final)
Monografia	Monograph/thesis
MonografiasParaCorrecao	N:N Monografia→Orientador (correction assignments)
Premonografia	Pre-project
SolicitacaoOrientacao	Advisor request (estudante→orientador)
Factura	Invoice for services
PagamentoPropina	Monthly tuition payment (with Multicaixa confirmation code)
NotaCobranca	Debt collection notice
ConfiguracaoTaxas	Singleton: global fee config (propina fallbacks per year, multa, taxa reenrollment, duracao_aula)
PrecoCurso	Per-course pricing (propina + multa by curricular year)
Servico	Dynamic services (nome, valor, activo)
Certificado	Certificate request/status
CertificadoDisciplinas	N:N Certificado→Disciplina
Declaracao	Academic declaration with QR verification code
HorarioAula	Weekly class schedule (curso, turno, dia, hora, sala)
PlanoProva	Exam calendar (PP1, PP2, Exame, Recurso, Especial)
PeriodoProva	Exam period date range
SistemaConfig	Singleton: system config (ano lectivo, matricula window, simulador,

Model	Description
	semestre_atual)
FaseAvaliacao	Evaluation phase control (AC, PP1, TTP/PP2, Exame, Recurso/Especial)
SnapshotSemestre	Semester snapshots for re-enrollment & finalization
AuditLog	Audit trail (quem, quando, IP, valor antes/depois)
AssinaturaGestor	Gestor signature image per departamento
AssinaturaPresidente	President signature image
AssinaturaDiretor	Director signature image
LayoutDocumento	Document layout templates (JSON content)
CurriculoAcademico	Student's academic curriculum per year
EstatisticasMonografiasDepartamento	Department monograph statistics

Nota Model (full structure)

```

model Nota {
  id_nota      Int      @id @default(autoincrement())
  id_estudante Int
  id_disciplina Int
  ano_lectivo  String    @db.VarChar(9)
  semestre     Semestre  @default(S1)

  // Avaliação Contínua
  ac1          Decimal?  @db.Decimal(4, 2)
  ac2          Decimal?  @db.Decimal(4, 2)
  ac3          Decimal?  @db.Decimal(4, 2)
  ttp          Decimal?  @db.Decimal(4, 2)
  pp1          Decimal?  @db.Decimal(4, 2)
  pp2          Decimal?  @db.Decimal(4, 2)

  // Exames
  exame        Decimal?  @db.Decimal(4, 2)
  recurso      Decimal?  @db.Decimal(4, 2)
  exame_especial Decimal? @db.Decimal(4, 2)

```



```
// Resultado
nota_final      Decimal?      @db.Decimal(4, 2)
dispensada      Boolean       @default(false)
tipo_avaliacao  TipoAvaliacao @default(Normal)
}
```

Grade Calculation Logic

```
nota_ac          = ((AC1 + AC2 + AC3) / 3 + TTP) / 2
nota_final_ac    = (nota_ac + (PP1 + PP2) / 2) / 2
```

```
Se tem_dispenza && nota_final_ac >= nota_dispenza
  → dispensada, nota_final = nota_final_ac
```

```
Se !dispensada && exame_lançado
  → nota_final = nota_final_ac × 40% + exame × 60%
```

```
Se recurso_lançado
  → nota_final = recurso (nota seca, máx 12)
```

```
Se exame_especial_lançado
  → nota_final = exame_especial (nota seca, máx 12)
```

nota_final é sempre calculada e guardada ao lançar notas — **nunca** calculada em runtime na API.

Edge cases do cálculo

- AC incompleto (ex: só AC1 e AC2 lançados, AC3 null) → não calcular **nota_final**, manter **null**
- **tem_dispenza = false** → ignorar threshold, ir sempre a exame independentemente da nota AC
- Recurso lançado sem exame prévio → bloquear na API (validação obrigatória)
- **exame_especial** lançado → substitui tudo, não combina com nada

8. Payment System

Sistema de Preços Hierárquico

Item	Escopo
Propina Mensal	● ESPECIFICO POR CURSO e ANO (via PrecoCurso)
Multa de Atraso	● ESPECIFICO POR CURSO e ANO
Taxa Monografia	● GLOBAL IGUAL PARA TODOS
Folhas de Prova	● GLOBAL IGUAL PARA TODOS
Taxa Rematrícula	● GLOBAL (via ConfiguracaoTaxas.taxa_reenrollment)

Lógica de busca (**lib/precos.ts**):

1. Sistema procura PRIMEIRO por preço definido especificamente para aquele curso e ano (**PrecoCurso**)
2. Se não encontrar valor definido, usa automaticamente o valor padrão global (**ConfiguracaoTaxas**)
3. Aplica desconto de bolsa se aplicável (50% ou 100%)

```
import { getPrecoEstudante } from "@lib/precos"
const { valor_propina, valor_com_desconto, valor_multa, tipo_bolsa, origem } =
await getPrecoEstudante(id_estudante)
```

Sistema de Bolsas

Tipo	Efeito
Nenhuma	Paga 100% do valor
Cinquenta	Paga 50% (metade) — sem multa de atraso
Cem	Isento — não paga nada, sem multa

Valores Padrão Globais (Fallback via ConfiguracaoTaxas)

Year	Monthly fee	Late fee
1st year	15,000 Kz	500 Kz
2nd year	20,000 Kz	500 Kz

Year	Monthly fee	Late fee
3rd year	25,000 Kz	500 Kz
4th year	30,000 Kz	500 Kz
5th year	35,000 Kz	500 Kz
6th year	40,000 Kz	500 Kz

Monthly Propina Flow

1. 1st of month → system generates payment record for every active student
2. Valor da propina é obtido automaticamente via `getPrecoEstudante()` com o valor correcto do curso
3. Student logs in → sees current month reference code and amount
4. If today > 10th and unpaid → late fee added automatically
5. Student takes reference to Multicaixa Express and pays
6. Student enters 3-digit confirmation code → system validates → flips to **Pago**

Reference Format

```
PROP-{YEAR}-{MONTH}-{STUDENT_INITIALS}-{3_DIGIT_CODE}
```

Example: PROP2026-04-BEN-847

Duplicate Payment Protection

Before confirming a payment, the API checks if a **Pago** record already exists for the same `mes + ano + id_estudante`. If so, returns **400** error.

Payment confirmation security

The confirmation endpoint uses `id_estudante` from the session, not from the request body.

What gets **BLOCKED** when payment is **Pendente**

- Cannot request an orientador
- Cannot download certificates
- Cannot submit monografia
- Cannot see grades for current year

What stays **ACCESSIBLE** when **Pendente**

- Login, profile, payment reference
- Grades from all previous years

Receptionist handles **ONLY**

- Caderneta payments, monografia fee, physical certificate pickup confirmation
- Multiple folhas de prova (with quantity selector modal)

Document Printing Limit

- **Documentos** (certificado de conclusão / declaração académica): **máximo 2 impressões por factura**
 - **Faturas**: sem limite de impressões
 - **Propinas**: sem limite de impressões
 - Cada impressão é registada no **AuditLog**
-

9. Business Logic Rules

Year Progression / Re-enrollment

- Student advances when **ALL** subjects of current year have **nota_final** ≥ 10 (max 2 failed subjects from any single curricular year)
- **Path A — Passou tudo**: avança para o ano seguinte + atribui novas disciplinas

- **Path B — Reprovou:** repete o ano, só as disciplinas falhadas são resetadas (notas anteriores preservadas em Snapshot)
- Snapshot realizado antes de qualquer mudança (`SnapshotSemestre`)
- Final year students cannot re-enroll (must pay monografia fee)
- Admin can manually suspend students who haven't re-enrolled (`suspenderEstudantesSemRematricula`)

Grade System

- **Scale:** 0–20 · **Passing:** 10+
- **Dispensa:** if `tem_dispensa = true` and `nota_final_ac >= nota_dispensa (14)` → dispensed from exam
- **Exam chain:** AC → (*if < 10 or no dispensa*) → Exame → (*if < 10*) → Recurso → (*if < 10*) → Exame Especial
- **Recurso / Exame Especial:** nota seca (raw score), maximum 12, replaces `nota_final` directly
- Every grade change writes to `AuditLog`

Grade Access Control

- **Orientador** → só lança/edita notas das disciplinas em `ProfessorDisciplina` com o seu `id_usuario` e o ano lectivo corrente
- **Gestor** → lança/edita notas de qualquer disciplina do seu departamento
- Nenhum dos dois pode editar notas de anos lectivos anteriores (validar na API)
- Bloqueio de notas para anos futuros: não permite lançar nota para disciplina cujo ano curricular > ano actual do estudante

Grade Computing (from CursoDisciplina)

O ano curricular e semestre das disciplinas vêm do `CursoDisciplina` (tabela que liga curso à disciplina), **não** dos campos fixos da tabela `Disciplina`. Uma disciplina pode estar no 2º ano do Curso A e no 3º ano do Curso B.

Average Calculation

- **Média por ano:** simple average of all subjects with `nota_final` — dispensadas count with `nota_final_ac`, em curso count with `nota parcial`
- **Média geral:** average of the yearly averages (only years with a calculated média)

Conflitos de Professor

Regra	Estado
Professor não pode leccionar duas disciplinas ao mesmo tempo (mesmo em cursos diferentes)	 Implementado
Apenas PP1 e PP2 verificados para conflitos (Exame/Recurso/Especial são vigiados por proctors)	 Implementado
Verificação cross-curso	 Implementado
Exclui apenas a mesma disciplina + curso + ano + semestre	 Implementado

Monografia Workflow

Submetida → EmRevisao → Aprovada → ParaDefender → Defendida
 ↓
 Rejeitada

- **Pré-projecto:** orientador NÃO é obrigatório (estudante pode submeter sem orientação)
- **Monografia:** orientador É obrigatório (só submete se tiver orientação aceite + propina paga)
- **Co-autor e Co-orientador:** campos opcionais, texto livre
- **Download:** orientador e gestor podem baixar o PDF
- Gestor agenda defesa e atribui nota final
- Pré-projecto Reprovado pode ser resubmetido

Monografia Access Rules

- Estudante só pode submeter se: `ano_current = 4` + orientação `Aceite` + propina `Pago`
- Orientador só revê monografias dos seus estudantes (`MonografiasParaCorrecao`)
- Gestor agenda defesa e atribui nota final de qualquer monografia do departamento

Horário de Aulas

- Turnos: Matinal (08:00-13:00), Vespertino (13:00-18:00), Noturno (18:00-23:00)
- Duração e intervalo configuráveis no Admin (`ConfiguracaoTaxas`)
- Cálculo automático de horários baseado na posição (1ª, 2ª, 3ª...)
- Unique constraint: `[id_curso, ano_curricular, semestre, turno, dia_semana, hora_inicio, ano_lectivo]`

Plano de Provas

- Tipos: PP1, PP2, Exame, Recurso, Exame_Especial
- Unique constraint: `[id_curso, id_disciplina, ano_curricular, semestre, tipo_prova, turno, ano_lectivo]`
- Timezone fix: backend usa `T12:00:00` (meio-dia) para evitar UTC shift

Sistema de Gestão de Administradores

- Nome de utilizador gerado automaticamente (primeiro nome + último sobrenome)
- Senha padrão: `admin123`
- Ações: reset password, remover administrador

Gestão de Orientadores Admin

- Barra de pesquisa + filtro por departamento + curso
- Ao marcar como gestor, verifica se já existe gestor no departamento
- Se existir, mostra modal de confirmação para substituir

Documentos / Assinaturas / Verificação

- Declarações académicas com código único de verificação (DECL-`{ano}`-`{numero}`-`{seq}`)
- Página pública `/verificar/[id]` para validar documentos
- Assinaturas: Presidente, Gestor (por departamento), Director — upload de imagem
- Layouts de documentos configuráveis via JSON

10. Test Credentials (after running seed)

Role	Email	Password	Contexto
Admin	admin@ispatlantida.ao	admin123	—
Student (2º ano)	estudante@ispatlantida.ao	student123	Ben — notas e pagamentos
Student (4º ano)	ana@ispatlantida.ao	student4ano123	Ana Silva — monografia e certificados
Student (4º ano)	carlos@ispatlantida.ao	student4ano123	Carlos Manuel — monografia (orientador: Prof. João)
Orientador	orientador@ispatlantida.ao	orientador123	Prof. Walter Neto — com funções de gestor (e_gestor=true)
Orientador 2 (NOT gestor)	orientador2@ispatlantida.ao	orientador123	Prof. João Mendes — sem funções de gestor
Receptionist	recepcao@ispatlantida.ao	recepcao123	—

Payment confirmation code: visible in terminal after seed or via `npx prisma studio` → table `PagamentoPropina`.

11. Seed Data

O sistema usa múltiplos ficheiros de seed para organização:

File	Purpose
<code>prisma/seed-base.ts</code>	Dados base: departamento, curso, admin, orientadores, recepcionista, preços
<code>prisma/seed-estudantes.ts</code>	Estudantes com notas, disciplinas, currículo académico
<code>prisma/seed-propinas.ts</code>	Propinas/mensalidades
<code>prisma/seed-registos.ts</code>	Registos académicos
<code>prisma/seed-semester.ts</code>	Configuração de semestre
<code>prisma/seed-rematricula.ts</code>	Rematrícula
<code>prisma/seed-snapshot-finalizados.ts</code>	Snapshot de estudantes finalizados
<code>prisma/seed.ts</code>	Seed principal que chama os seeds acima

Run with: `npx tsx prisma/seed.ts`

Currently creates

- 1 department, 1 course (Engenharia Informática, 4 anos)
- 6 propina fallback prices (one per year)
- 15+ subjects across 4 years
- 1 admin, 2 orientadores (1 with `e_gestor=true`), 1 recepcionista
- Multiple students with different profiles

All seeds use `upsert` — safe to re-run without duplicating data.

12. Commands Reference

```
# Start development server
npm run dev

# After every schema change – always run both:
npx prisma db push
npx prisma generate

# Run seeder
npx tsx prisma/seed.ts

# Open Prisma Studio (visual database editor)
npx prisma studio

# Reset database completely and re-seed
npx prisma migrate reset
npx prisma db push
npx tsx prisma/seed.ts
```

13. Module Build Plan

Module 1 — Student Portal COMPLETO

Visão Geral (/estudante)

-  Perfil: nome, curso, ano actual, estado da propina
-  Seletor de ano/semestre com disciplinas da BD
-  Banner de propina pendente com referência e valor
-  API GET </api/estudante/perfil>
-  API GET [api/estudante/disciplinas?ano=&semestre=](/api/estudante/disciplinas?ano=&semestre=)

Notas (/estudante/notas)

-  Tabela agrupada por Ano → Semestre
-  Badge: Aprovado / Reprovado / Dispensado / Em Curso
-  Nota parcial a amarelo para disciplinas em curso
-  Painel expandido: componentes AC + cadeia de exames com setas
-  Nota seca identificada para recurso e especial
-  Média por ano + Média geral
-  Notas do ano corrente bloqueadas se propina pendente
-  API GET </api/estudante/notas>

Pagamentos (/estudante/pagamentos)

- ☒ Resumo: total em dívida, pagamentos pendentes, pagamentos efectuados
- ☒ Histórico completo ordenado
- ☒ Estado: Pendente / Atrasado / Pago com cores
- ☒ Valor base + multa separados
- ☒ Referência Multicaixa + código de 3 dígitos
- ☒ Protecção contra pagamento duplicado
- ☒ API GET /api/estudante/pagamentos
- ☒ API POST /api/estudante/pagamentos/confirmar

Monografia (/estudante/monografia)

- ☒ Submeter pré-projecto e monografia
- ☒ Upload de PDF (máx 10MB)
- ☒ Co-autor e Co-orientador
- ☒ API GET /api/estudante/monografia
- ☒ API POST /api/estudante/monografia/upload
- ☒ API POST /api/estudante/premonografia

Certificados (/estudante/certificados)

- ☒ Lista de certificados + pedir novo
- ☒ Download PDF
- ☒ Certificado de Disciplinas + Conclusão
- ☒ API GET /api/estudante/certificados
- ☒ API POST /api/estudante/certificados/pedir

Horário (/estudante/horario)

- ☒ Grade semanal do estudante (por turno)
- ☒ Disciplinas pendentes de anos anteriores
- ☒ Nome do professor por disciplina
- ☒ API GET /api/estudante/horario

Serviços (/estudante/servicos)

- ☒ Comprar serviços (declarações, etc.)
- ☒ Confirmar pagamento

- ☒ API GET `/api/estudante/servicos`
 - ☒ API POST `/api/estudante/servicos/confirmar`
-

Module 2 — Grade Management

COMPLETO

- ☒ Modelo `ProfessorDisciplina` no schema
- ☒ Seed com disciplinas atribuídas (BD1, RC1, SO1)
- ☒ `lib/audit.ts` — helper `logAudit()`
- ☒ `lib/notas.ts` — `calcularNotaFinal()`
- ☒ Dashboard do orientador com edição inline
- ☒ Dashboard do gestor com visão geral
- ☒ `/gestor/estudantes` + `/gestor/disciplinas`
- ☒ Edição inline de notas com pré-visualização
- ☒ Filtros por curso, ano, semestre, turno
- ☒ Gestão de professores responsáveis (1 por disciplina)
- ☒ Conflitos de professor (horário + provas)
- ☒ Correção: ano/semestre vindo do `CursoDisciplina` (não da `Disciplina`)
- ☒ Bloqueio de notas para anos futuros

Plano Escolar (Horário + Provas)

- ☒ `/gestor/horario` — grade semanal com turno, impressão
 - ☒ `/gestor/plano-provas` — calendário de provas, impressão
 - ☒ `/gestor/curriculo` — gestão do currículo
 - ☒ API CRUD completo para horário, provas, período
 - ☒ Turnos configuráveis no curso
 - ☒ Duração/intervalo configurável no admin
-

Module 3 — Monografia Workflow

COMPLETO

Estudante

- ☒ `/estudante/orientador` — pedir orientação

- ☒ Ver estado do pedido
- ☒ API **POST/GET** `/api/estudante/solicitacao-orientacao`

Orientador

- ☒ `/orientador/solicitacoes` — aceitar/rejeitar
- ☒ `/orientador/monografias` — rever, feedback
- ☒ APIs completas

Gestor

- ☒ `/gestor/monografias` — agendar defesa, nota final
 - ☒ Pipeline visual de monografias
 - ☒ APIs completas
-

Module 4 — Recepcionista ☒ COMPLETO

- ☒ `/recepcionista` — pesquisa de estudante (debounce)
 - ☒ `/recepcionista/estudante/[id]` — detalhe completo
 - ☒ Confirmar levantamento de certificado
 - ☒ Registrar pagamento de folhas de prova (quantidade múltipla)
 - ☒ Registrar pagamento de taxa de monografia
 - ☒ Limite de 2 impressões por documento
 - ☒ APIs completas
-

Module 5 — Admin ☒ 100% COMPLETO

- ☒ Dashboard com stats
- ☒ Gestão de Estudantes (CRUD + notas + pagamentos)
- ☒ Gestão de Orientadores (pesquisa, filtros, gestor)
- ☒ Gestão de Recepcionistas
- ☒ Gestão de Administradores
- ☒ Gestão de Cursos (com preços por ano)
- ☒ Gestão de Disciplinas
- ☒ Gestão de Departamentos (com gestor)
- ☒ Gestão de Pagamentos

- ☒ Preços Globais + Serviços + Horário
 - ☒ Audit Log (filtros avançados, cartões de contexto)
 - ☒ Sistema: Ano Lectivo, Semestre, Finalistas, Layout Documentos, Assinaturas, Registos
 - ☒ Simulador de data do sistema
-

14. Extra Features (Post-Completion)

Rematrícula

- ☒ `lib/reenrollment.ts` — `processarRematricula()` com snapshot
- ☒ Avanço de ano (com atribuição automática de disciplinas)
- ☒ Repetição de ano (reset só das disciplinas falhadas)
- ☒ Suspensão de estudantes sem rematrícula
- ☒ `SnapshotSemestre` para preservar dados antes da mudança

Documentos e Assinaturas

- ☒ Declarações académicas com código de verificação
- ☒ Página pública `/verificar/[id]`
- ☒ Assinaturas: Presidente, Gestor (por departamento), Director
- ☒ Layouts de documentos configuráveis (JSON)
- ☒ PDFs com `@react-pdf/renderer`

Configuração de Preços por Curso

- ☒ `PrecoCurso` — preço de propina e multa por curso + ano
- ☒ Admin pode configurar no modal de edição de cursos
- ☒ Fallback para valores globais se não houver específico

Serviços Dinâmicos

- ☒ `Servico` — nome, descrição, valor, activo
- ☒ Admin pode criar/editar/desactivar serviços

- ☒ Estudante compra e confirma pagamento

Sistema de Bolsas

- ☒ **TipoBolsa**: Nenhuma, Cinquenta (50%), Cem (100%)
- ☒ Isenção de multa para bolseiros
- ☒ Desconto automático no cálculo da propina

Gestão Centralizada (Admin + Gestor)

- ☒ **app/admin/adminNav.ts** — fonte única de navegação admin
- ☒ **app/gestor/gestorNav.ts** — navegação gestor com dropdown

15. Component Architecture

DashboardLayout (shared by all roles)

Props: **navItems**, **title**, **subtitle?**, **children** **Contains:** Sidebar (brand + nav + logout) · Topbar · Content area Active nav item detected via **usePathname()** — highlights automatically

☒ Detecta automaticamente sessão expirada e redireciona para login.

Pattern for each page

<code>app/[role]/page.tsx</code>	→ server component, auth check, renders Dashboard
<code>app/[role]/[Role]Dashboard</code>	→ client component, fetches from API, renders UI
<code>app/api/[resource]/route.ts</code>	→ auth check → role check → prisma query → JSON

AuditLog helper (lib/audit.ts)

```
export async function logAudit({ id_usuario, acao, tabela, id_registro,
valor_antes, valor_depois, ip_address }) {
```

```

await prisma.auditLog.create({
  data: {
    id_usuario, acao, tabela, id_registro,
    valor_antes: valor_antes ? JSON.stringify(valor_antes) : null,
    valor_depois: valor_depois ? JSON.stringify(valor_depois) : null,
    ip_address, criado_em: new Date()
  }
})
}

```

Grade calculation helper (lib/notas.ts)

```

export function calcularNotaFinal(nota: NotaInput, disciplina: DisciplinaInput) {
  const { ac1, ac2, ac3, ttp, pp1, pp2, exame, recurso, exame_especial } = nota

  // Exame especial substitui tudo
  if (exame_especial !== null) return { nota_final: exame_especial, tipo: "Especial" }

  // Recurso substitui tudo
  if (recurso !== null) return { nota_final: recurso, tipo: "Recurso" }

  // AC incompleto – não calcular ainda
  if ([ac1, ac2, ac3, ttp, pp1, pp2].some(v => v === null))
    return { nota_final: null, tipo: "Normal" }

  const nota_ac = (((ac1! + ac2! + ac3!) / 3) + ttp!) / 2
  const nota_final_ac = (nota_ac + (pp1! + pp2!) / 2) / 2

  // Dispensa
  if (disciplina.tem_dispensa && nota_final_ac >= disciplina.nota_dispensa)
    return { nota_final: nota_final_ac, dispensada: true, tipo: "Normal" }

  // Com exame
  if (exame !== null) return { nota_final: nota_final_ac * 0.4 + exame * 0.6, tipo: "Normal" }

  // AC completo mas sem exame
  return { nota_final: null, tipo: "Normal" }
}

```

16. UI Design System

Token	Value
Background	#0d0f14
Card	#1e2230

Token	Value
Sidebar	#13161e
Primary	#e03d3d (red)
Secondary	#f0a500 (gold)
Teal	#2dd4bf
Text	#e8eaf0
Muted	#9098b0
Dim	#555e78
Border	rgba(255,255,255,0.07)
Font	system-ui

Status Colours

State	Colour
Pass / Paid / Active	#22c55e
Pending / Warning / Nota parcial	#f0a500
Fail / Late / Error	#e03d3d
Dispensado / Info	#2dd4bf
Em curso / Sem nota	#555e78
Exame Especial	#9b59b6

17. Dificuldades e Lições Aprendidas

Next.js 15 App Router — **params** é uma Promise

Todos os endpoints API que recebiam `params.id` deixaram de funcionar pois agora `params` é uma `Promise`.

✓ **Solução:** Adicionar `const resolvedParams = await params` ANTES de aceder a qualquer propriedade.

❌ Prisma Decimal — Bug `Number(null) = 0`

`Number(null)` retorna 0 em JavaScript, que é uma nota válida.

✓ **Solução:** Sempre verificar com `!= null` ANTES de converter para `Number`.

❌ TurboPack Hydration Errors

O novo bundler do Next.js 15 causa vários erros de hidratação com datas e formatação.

✓ **Solução:** Remover fontes padrão Geist, formatar datas apenas dentro de `useEffect()`.

❌ Timezone UTC shift (Africa/Luanda UTC+1)

`new Date("2025-03-17T00:00:00")` em UTC+1 vira 23:00 UTC do dia anterior.

✓ **Solução:** Usar `T12:00:00` (meio-dia) em vez de `T00:00:00` para evitar shift.

❌ Sidebar nav overflow

Quando a sidebar tinha muitos itens, o logout ficava inacessível.

✓ **Solução:** `height: 100vh + overflow: hidden` no aside, `overflow-y: auto` no nav.

❌ JWT não atualiza após edição de perfil

Disciplinas aparecem no ano/semestre errado

O código usava `Disciplina.ano_curricular` em vez de `CursoDisciplina.ano_curricular`.

✅ **Solução:** Corrigido em 6 ficheiros para usar valores do `CursoDisciplina`.

Edição de notas — valores sobrescritos com null

✅ **Solução:** API PUT ignora null, frontend faz fetch directo após save.

18. Current State

Completed

- ☒ Full project scaffolded (Next.js 15 + TypeScript + Tailwind)
- ☒ PostgreSQL database with 37 models + 17 enums
- ☒ Production deploy on Vercel + Neon: sge-atl-ntida.vercel.app
- ☒ NextAuth JWT authentication + refresh-jwt + middleware
- ☒ All 5 role dashboards (Admin, Gestor, Orientador, Recepcionista, Estudante)
- ☒ Student: perfil, notas, pagamentos, monografia, certificados, horário, serviços
- ☒ Orientador: disciplinas, solicitações, monografias, download
- ☒ Gestor: estudantes, disciplinas, notas, horário, plano provas, currículo, monografias, pipeline
- ☒ Recepcionista: pesquisa, emitir, auditar impressão
- ☒ Admin: CRUD completo + sistema + audit + preços + assinaturas + documentos
- ☒ Rematrícula com snapshot e suspensão
- ☒ Sistema de bolsas (50% e 100%)
- ☒ Conflitos de professor
- ☒ Verificação de documentos com código único
- ☒ Assinaturas digitais (presidente, gestor, diretor)

- ☒ Audit logging completo em todas as operações
- ☒ QR code verification on certificates and declarations
- ☒ Production-ready with Vercel + Neon (QR codes, HTTPS, login fix)